Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	02-03-2026		
Name	Priya Verma - 3122235001102		

Experiment 6

Decision Tree and Random Forest: A Comparative Classification Study

1 Objective

- To implement a **Decision Tree classifier**.
- To extend the Decision Tree into a **Random Forest ensemble model**.
- To study the impact of hyperparameters on overfitting and generalization.
- To **select optimal hyperparameters using 5-Fold Cross-Validation**.
- To compare single-tree and ensemble-tree models.

2 Dataset Description

Wisconsin Diagnostic Breast Cancer Dataset

- Total samples: 569
- Features: 30 numerical attributes
- Target classes: Malignant (M) and Benign (B)

3 Implementation Details

The following code snippets illustrate the core implementation of the classifiers:

Important Libraries to be included

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as snsr

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV,
    cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report,
    roc_curve, auc
)

plt.rcParams.update({
    "font.family": "Arial",
    "font.size": 15,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "axes.labelsize": 15,
    "axes.labelweight": "bold",
    "xtick.labelsize": 13,
    "ytick.labelsize": 13
})
```

1. Load the dataset and encode class labels.

```
data = load_breast_cancer()

X = data.data
y = data.target   # Already encoded (0 = malignant, 1 = benign)

feature_names = data.feature_names
target_names = data.target_names

print("Dataset Shape:", X.shape)
print("Target Classes:", target_names)
```

2. Perform Exploratory Data Analysis

```
# Class Distribution
plt.figure()
sns.countplot(x=y)
plt.title("Class Distribution")
plt.show()

# Correlation Heatmap
df = pd.DataFrame(X, columns=feature_names)
df['target'] = y

# Compute correlation matrix
```

```
corr_matrix = df.corr() * 100   # Convert to percentage

plt.figure(figsize=(12,10))
sns.heatmap(
    corr_matrix,
    cmap='coolwarm',
    annot=True,           # Show values
    fmt=".1f",            # 1 decimal place
    annot_kws={"size":6}
)

plt.title("Feature Correlation Heatmap (%)")
plt.show()
```

3. Split the dataset into training and testing sets (80–20)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

4. Train a Decision Tree classifier.

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
```

5. Define a hyperparameter search space.

```
dt_param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 3, 5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

6. Perform 5-Fold Cross-Validation to evaluate each hyperparameter combination.

```
dt_grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    dt_param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=1
)

dt_grid.fit(X_train, y_train)
bestParam_DT = dt_grid.best_params_
```

```
acc_scores_dt = cross_val_score(dt, X_train, y_train, cv=5, scoring='accuracy')
f1_scores_dt = cross_val_score(dt, X_train, y_train, cv=5, scoring='f1')

print("Best Parameters (Decision Tree):", bestParam_DT)
print("Best CV Accuracy:", dt_grid.best_score_)

dt_table = pd.DataFrame({
    "Criterion": [bestParam_DT["criterion"]],
    "Max Depth" : [bestParam_DT["max_depth"]],
    "Min_samples_leaf":[bestParam_DT["min_samples_leaf"]],
    "Min_samples_split": [bestParam_DT["min_samples_split"]],
    "Average CV Accuracy %": [acc_scores_dt.mean()*100],
    "Average CV f1 Score" : [f1_scores_dt.mean()]
})
dt_table
```

	Criterion	Max Depth	Min_samples_leaf	Min_samples_split	Average CV Accuracy %	Average CV f1 Score
0	gini	5	1	5	90.989011	0.928176

Figure 1: Best Parameters for Decision Tree using 5-Fold Cross Validation

7. Select the hyperparameters that yield the best average cross-validation performance.

```
# Best Model
best_dt = dt_grid.best_estimator_
print(best_dt)

# Output
# DecisionTreeClassifier(max_depth=5, min_samples_split=5, random_state=42)
```

8. Train a Random Forest classifier.

```
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
```

9. Repeat cross-validation-based hyperparameter selection.

```
rf_param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'max_features': ['sqrt', 'log2'],
    'bootstrap': [True, False]
}

rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    rf_param_grid,
```

```
    cv=5,
    scoring='accuracy',
    n_jobs=1
)

rf_grid.fit(X_train, y_train)
bestParam_RF = rf_grid.best_params_

print("Best Parameters (Random Forest):", bestParam_RF)
print("Best CV Accuracy:", rf_grid.best_score_)
acc_scores_rf = cross_val_score(rf, X_train, y_train, cv=5, scoring='accuracy')
f1_scores_rf = cross_val_score(rf, X_train, y_train, cv=5, scoring='f1')

rf_table = pd.DataFrame({
    "n_estimators": [bestParam_RF["n_estimators"]],
    "Max Depth" : [bestParam_RF["max_depth"]],
    "Max Features" : [bestParam_RF["max_features"]],
    "Boostrap": [bestParam_RF["bootstrap"]],
    "Average CV Accuracy %": [acc_scores_rf.mean()*100],
    "Average CV f1 Score" : [f1_scores_rf.mean()]
})
rf_table
```

	n_estimators	Max Depth	Max Features	Boostrap	Average CV Accuracy %	Average CV f1 Score
0	100	None	log2	False	95.384615	0.963408

Figure 2: Best Parameters for Random Forest

10. Compare both models using evaluation metrics.

```
from sklearn.metrics import roc_auc_score

# Probabilities for positive class (class = 1)
y_prob_dt = best_dt.predict_proba(X_test)[:, 1]
y_prob_rf = best_rf.predict_proba(X_test)[:, 1]

# ROC AUC scores
roc_auc_dt = roc_auc_score(y_test, y_prob_dt)
roc_auc_rf = roc_auc_score(y_test, y_prob_rf)

evaluate = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "Accuracy": [accuracy_score(y_test, y_pred_dt), accuracy_score(y_test,
    y_pred_rf)],
    "Precision": [precision_score(y_test, y_pred_dt), precision_score(y_test,
    y_pred_rf)],
    "Recall": [recall_score(y_test, y_pred_dt), recall_score(y_test, y_pred_rf)],
    "F1 Score": [f1_score(y_test, y_pred_dt), f1_score(y_test, y_pred_rf)],
    "ROC AUC": [roc_auc_dt, roc_auc_rf]
})

evaluate
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	Decision Tree	0.921053	0.956522	0.916667	0.936170	0.916336
1	Random Forest	0.956140	0.958904	0.972222	0.965517	0.994378

Figure 3: Compare both models.

4 Visualizations

This section presents the visual analysis of the class distribution, feature correlation analysis, confusion matrix and ROC Curve.

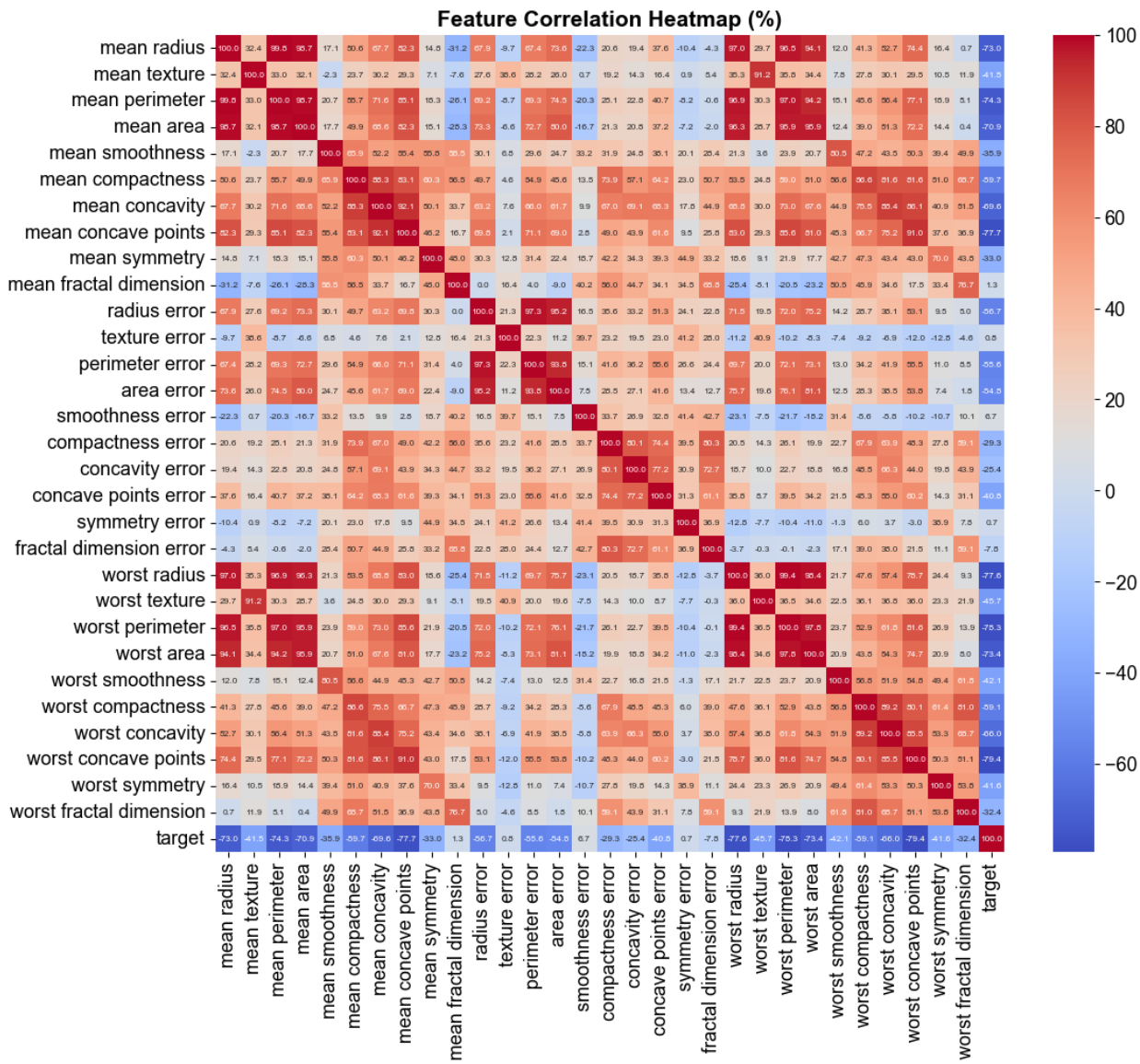

Figure 4: Feature correlation analysis

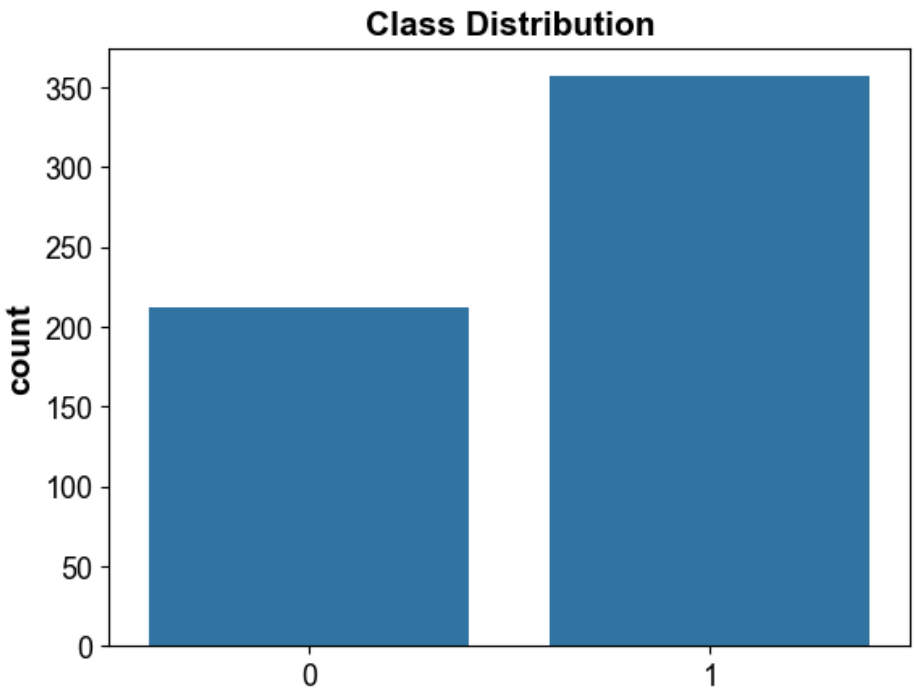

Figure 5: Class Distribution Plots

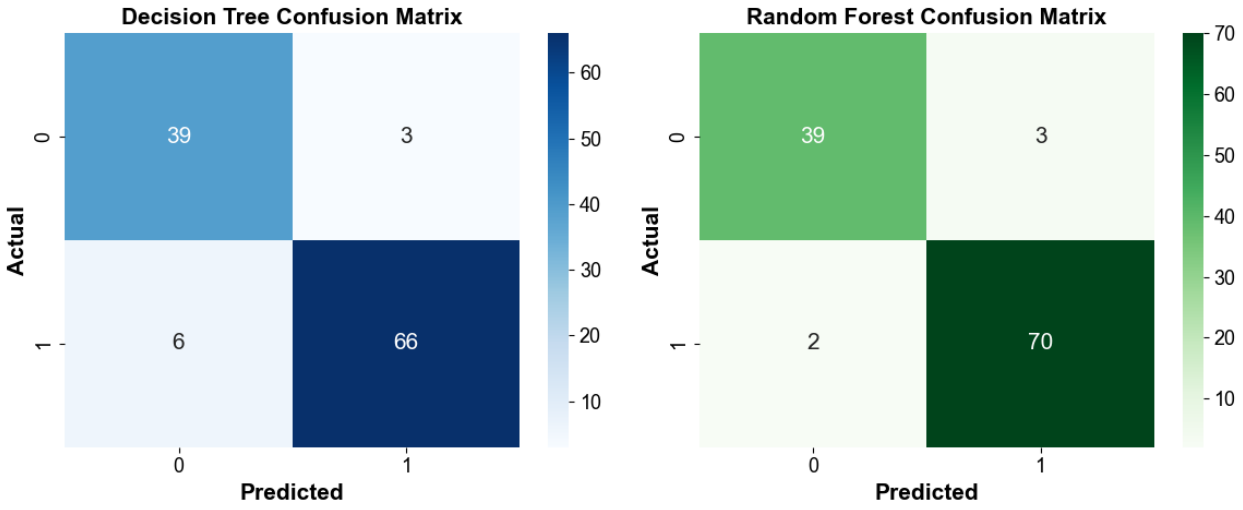

Figure 6: Confusion Matrix

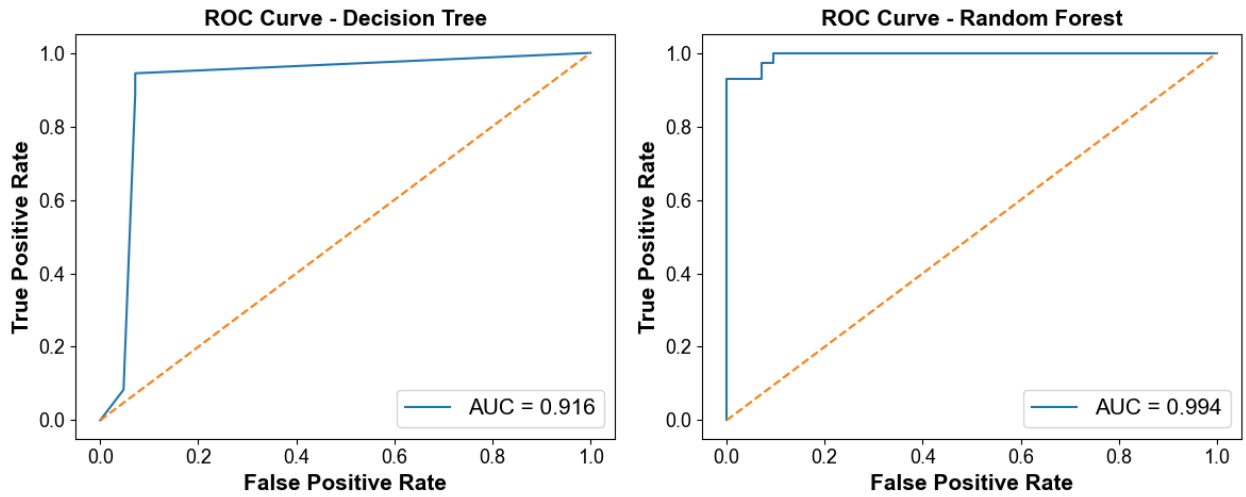

Figure 7: ROC Curve

5 Performance Tables

Table 1: Decision Tree Hyperparameter Evaluation using 5-Fold Cross-Validation

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
gini	5	90.98	0.93

Table 2: Random Forest Hyperparameter Evaluation using 5-Fold Cross-Validation

n_estimators	Max Depth	Max Features	Avg CV Accuracy (%)	Avg CV F1 Score
100	None	log2	95.38	0.96

Table 3: 5-Fold Cross-Validation Accuracy Comparison

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	0.94	0.94	0.90	0.94	0.95	0.94
Random Forest	0.97	1.00	0.93	0.95	0.97	0.97

6 Observations

- **How does tree depth affect overfitting in Decision Trees?**

  Tree depth controls model complexity. A very deep tree learns noise from training data and causes overfitting, while a smaller depth improves generalization. In this experiment, max_depth = 5 gave the best performance by balancing bias and variance.

- **Which hyperparameter had the greatest impact on performance?**

The most impactful hyperparameter was max_depth because it directly controls model complexity and overfitting. Other parameters like min_samples_split had less impact.

- **How does Random Forest improve generalization?**

  Random Forest improves generalization by training multiple trees on different data samples and averaging their predictions. This reduces variance, prevents overfitting, and gives better accuracy than a single Decision Tree.

- **Did ensemble learning always improve performance? Why or why not?**

  No, ensemble learning does not always improve performance. Its effectiveness depends on data quality, model tuning, and problem complexity. However, in this experiment, Random Forest performed better by reducing overfitting.

7 Conclusion

- Decision Tree and Random Forest models were implemented and evaluated using 5-fold cross-validation.
- Hyperparameters were selected based on average cross-validation performance, ensuring robust generalization.
- The results demonstrate that Random Forest reduces variance and improves stability compared to a single Decision Tree.